

Investigación aplicada sobre estrategias de testing automatizado en desarrollo de software

Andrés Mauricio Noscue Cerquera
Servicio Nacional de Aprendizaje (SENA)
Neiva, Colombia
mauronoscue@gmail.com

Resumen

Este artículo presenta un estudio de sobre testing y automatización de procesos ,muchos sistemas de agendamiento en salud fracasan no por falta de funciones, sino por errores técnicos que frustran al usuario que interactúa con el mismo . Este artículo examina cómo blindar la calidad de una plataforma de gestión de citas aplicando estrategias de validación automática. Para ello, se realizó una revisión de la literatura reciente que vincula la ingeniería de pruebas con la fiabilidad del software. Los textos analizados coinciden en que depender de revisiones manuales deja brechas críticas en la seguridad y la corrección del código. La investigación toma estos hallazgos específicamente la capacidad de detectar fugas de memoria y mejorar la usabilidad móvil y propone un modelo de integración para el sistema de citas. Se argumenta que adoptar ciclos de entrega continua y pruebas dinámicas es la única vía para asegurar que la plataforma soporte la carga real de usuarios. En suma, el estudio demuestra que la arquitectura del software médico debe priorizar la automatización de controles para garantizar un servicio estable.

Keywords

testing automatizado, buenas prácticas, ingeniería de software, reproducibilidad, investigación aplicada

1. Introducción

La estabilidad operativa de las plataformas digitales en el sector salud se sostiene sobre una infraestructura técnica extremadamente delicada e invisible para el usuario final. Más allá de la interfaz y la funcionalidad observable por el paciente o el profesional, el software clínico inevitablemente acumula formas de desgaste operativo, manifestándose como fugas de memoria persistentes, saturación progresiva de recursos internos (backend y bases de datos), y fallos complejos de concurrencia que la inspección manual y el testing superficial son incapaces de detectar oportunamente. Esta situación revela una desconexión crítica en la industria: mientras la complejidad inherente a los servicios sanitarios se expande continuamente para abarcar la telemedicina, la monitorización remota y los modelos híbridos de atención, los métodos tradicionales de validación y aseguramiento de calidad (QA) continúan dependiendo en exceso del criterio humano. Esta dependencia se traduce en una vulnerabilidad inherente al error por fatiga y carece de la escala y la velocidad necesarias para contener la regresión del código tras cada iteración de desarrollo. La literatura técnica contemporánea es clara al sugerir que la defensa más efectiva frente a este deterioro progresivo es la automatización disciplinada de las pruebas y los procesos de QA, permitiendo ciclos de validación repetibles y a gran escala. De hecho, estudios de ingeniería de sistemas críticos

indican que métricas esenciales para la fiabilidad, como el Mean Time to Overflow (MTOF) —el tiempo promedio antes del colapso de un recurso crítico—, solo pueden estimarse de forma precisa mediante scripts capaces de estresar el sistema simulando las condiciones de carga máxima antes del despliegue productivo. No obstante, el panorama actual en HealthTech dista de ser ideal: aunque las herramientas de automatización demuestran gran eficacia al validar la funcionalidad básica y los flujos de trabajo definidos, persisten vacíos significativos en dimensiones cruciales para el ciclo de vida a largo plazo del software, incluyendo la portabilidad de los scripts a través de diversos entornos, la mantenibilidad del código de prueba para que no se vuelva frágil (brittle) y, fundamentalmente, la capacidad de la arquitectura de testing para garantizar que el software continúe siendo legible, modificable y auditable durante los largos periodos requeridos por las regulaciones sanitarias.

2. Marco teórico y trabajos relacionados

La calidad del software en el sector salud demanda un examen técnico continuo. La estabilidad, la seguridad y la integridad de los datos son requisitos innegociables, por lo que el desarrollo debe asegurar la predictibilidad del sistema. Los métodos basados en la revisión manual resultan insuficientes ante la escala actual de sistemas complejos, los cuales gestionan cargas multicanal y flujos de trabajo de alta exigencia.

Una referencia esencial para medir el desempeño técnico proviene de la investigación que articuló las métricas DORA. Estos indicadores (frecuencia de entrega, *lead time* y tasas de fallos) trascienden la evaluación de la velocidad del equipo: funcionan como un barómetro de la resiliencia sistémica. En contextos clínicos, donde la interrupción del servicio es inadmisible, la medición de estabilidad y tiempo de recuperación adquiere una relevancia crítica.

La ingeniería robusta se sostiene sobre el postulado de que las pruebas deben anteceder a la producción del código (TDD). Este principio promueve la simplicidad y garantiza un diseño modular. Su relación directa con la Integración Continua (CI) es evidente, dado que la fusión constante de versiones reduce el riesgo acumulado de fallos, aspecto vital para sistemas con alta criticidad temporal.

No obstante, la automatización no constituye una solución completa en sí misma. Diversos análisis empíricos muestran que su efectividad está fuertemente condicionada por la cultura del equipo y las capacidades internas. La adopción de estas metodologías sin abordar la resistencia al cambio o sin modernizar arquitecturas heredadas suele derivar en impactos marginales. La eficiencia, por tanto, depende de la coordinación rigurosa entre herramientas y estructura organizacional.

Otro pilar fundamental es la evolución del código fuente, entendida como mantenibilidad. Los esfuerzos en validación automática

y ciclos de entrega pierden sentido si la estructura del software no es coherente o dificulta la legibilidad. En el sector salud, donde la adaptación normativa y la escalabilidad son demandas constantes, este componente se convierte en un requisito de supervivencia técnica.

Aunque el conocimiento técnico acumulado es amplio, persisten brechas de investigación. Se requieren estudios que integren de manera sistemática prácticas como CI, TDD y automatización, con el fin de cuantificar sus beneficios combinados en escenarios de alta criticidad. También es necesario analizar cómo estas metodologías se adaptan a dominios caracterizados por demanda fluctuante, en los que el costo de una interrupción es asistencial y operativamente inaceptable.

3. Metodología de investigación aplicada

3.1. Diseño

Se eligió un diseño de investigación-acción (*action research*), un modelo apropiado para entornos donde la calidad debe evaluarse a la par del comportamiento operativo. Este enfoque metodológico tiene la ventaja de permitir intervenciones directas en el proceso mientras se documenta su impacto, lo que resulta especialmente valioso en proyectos de desarrollo de software donde la teoría y la práctica deben informarse mutuamente de forma continua. La metodología se articuló en ciclos continuos de intervención, medición y reflexión sobre el proceso de desarrollo, documentando el efecto de introducir prácticas de automatización en el ecosistema sanitario digital.

La elección de investigación-acción no fue arbitraria. Este diseño se alinea naturalmente con la naturaleza iterativa del desarrollo de software y permite que los hallazgos de cada ciclo informen las decisiones del siguiente. A diferencia de métodos puramente observacionales, la investigación-acción reconoce que el investigador es también un participante activo que introduce cambios deliberados en el sistema bajo estudio. Esta característica fue particularmente relevante para el proyecto, donde no se trataba simplemente de observar cómo funcionaban las prácticas existentes, sino de implementar mejoras y medir su impacto en tiempo real.

El trabajo se complementó con un estudio de caso técnico enfocado en determinar cómo las distintas estrategias de validación —pruebas automáticas, monitoreo continuo y análisis de regresiones— influyen directamente en la estabilidad y mantenibilidad del código fuente. El estudio de caso proporcionó el contexto específico necesario para anclar los hallazgos en una realidad concreta, evitando generalizaciones excesivas que podrían no aplicar a otros contextos. Al mismo tiempo, la naturaleza detallada del estudio de caso permitió identificar matices y relaciones causales que habrían sido invisibles en un análisis más superficial o generalizado.

Se establecieron formalmente hipótesis de trabajo y criterios claros de éxito, centrados en la reducción de defectos, la fiabilidad y la robustez del sistema bajo escenarios de alta carga. Las hipótesis no solo guiaron la recolección de datos, sino que también proporcionaron un marco para interpretar los resultados y determinar si las intervenciones habían sido efectivas. Los criterios de éxito se definieron de manera operacional, con umbrales cuantitativos específicos que permitían determinar objetivamente si se habían alcanzado las mejoras esperadas. Esta formalización fue crucial para

mantener el rigor del proceso y evitar interpretaciones subjetivas de los resultados.

Además, el diseño incluyó mecanismos para capturar tanto datos cuantitativos como observaciones cualitativas. Si bien las métricas numéricas proporcionaban evidencia objetiva de mejoras o deterioros en la calidad del sistema, las observaciones cualitativas del equipo capturaban aspectos más sutiles como cambios en la confianza, la velocidad percibida de desarrollo o las dificultades encontradas al adoptar nuevas prácticas. Esta triangulación de fuentes de evidencia fortaleció la validez de las conclusiones al proporcionar múltiples perspectivas sobre el mismo fenómeno.

3.2. Comparación de metodologías ágiles

Para la selección del enfoque de trabajo más adecuado, se llevó a cabo una evaluación multi-criterio de metodologías ágiles ampliamente utilizadas, como Scrum, Kanban y Programación Extrema (XP). La comparación no se limitó a una revisión teórica de las características de cada metodología, sino que consideró específicamente cómo cada una se ajustaba a las necesidades particulares del proyecto y del contexto organizacional en el que se desarrollaba. Se reconoció desde el inicio que no existe una metodología universalmente superior, sino que la elección debe basarse en la compatibilidad entre las características de la metodología y las demandas del proyecto específico.

La comparación se centró en factores críticos para el contexto del proyecto: la velocidad de la iteración, el soporte para incorporar herramientas de automatización y la capacidad de adaptarse a cambios funcionales. La velocidad de iteración era importante porque el proyecto necesitaba entregar valor de forma frecuente a los usuarios y recibir retroalimentación temprana que informara el desarrollo subsecuente. El soporte para automatización era crucial porque uno de los objetivos principales era integrar prácticas de aseguramiento de calidad automatizado en el flujo de trabajo. Y la capacidad de adaptación era esencial porque, como ocurre en muchos proyectos de software, los requisitos evolucionaban a medida que se comprendía mejor el dominio del problema y las necesidades de los usuarios.

Scrum destacó por su estructura de sprints de duración fija, que proporcionaba un ritmo predecible y puntos de reflexión regulares a través de las retrospectivas. Esta cadencia facilitaba la introducción gradual de nuevas prácticas de automatización, permitiendo que el equipo experimentara con una herramienta o técnica durante un sprint y evaluara sus resultados antes de decidir si adoptarla permanentemente. Además, las ceremonias de Scrum, especialmente la planificación y la revisión de sprint, proporcionaban oportunidades naturales para discutir aspectos de calidad y decidir cuánto esfuerzo dedicar a mejoras técnicas versus nuevas funcionalidades.

Kanban, por otro lado, ofrecía mayor flexibilidad al no prescribir iteraciones de duración fija, lo que podría haber sido ventajoso en un entorno con prioridades altamente volátiles. Sin embargo, esta misma flexibilidad también significaba menos estructura para momentos de reflexión y planificación, lo que podría haber dificultado la introducción sistemática de prácticas de calidad. La naturaleza de flujo continuo de Kanban funcionaba mejor en contextos donde las tareas eran relativamente homogéneas y el trabajo llegaba de

forma constante, lo cual no era exactamente el caso de este proyecto.

La Programación Extrema (XP) enfatizaba fuertemente prácticas técnicas como el desarrollo guiado por pruebas (TDD), la integración continua y la programación en pares. Estas prácticas estaban muy alineadas con los objetivos de calidad del proyecto. Sin embargo, XP requería un nivel de disciplina técnica y compromiso del equipo que podría haber sido difícil de implementar completamente, especialmente al inicio del proyecto cuando el equipo todavía estaba desarrollando sus capacidades en automatización. Además, algunas prácticas de XP como la programación en pares requerían ajustes culturales y logísticos que habrían tomado tiempo en implementarse efectivamente.

El análisis concluyó que la estructura iterativa de Scrum ofrecía el equilibrio óptimo entre la flexibilidad requerida para entregas continuas y la frecuencia necesaria para la introducción gradual de la verificación automática y los ciclos de experimentación. Scrum proporcionaba suficiente estructura para mantener el foco y la disciplina, pero sin ser tan prescriptivo que inhibiera la adaptación o la experimentación. Además, Scrum era familiar para algunos miembros del equipo, lo que reducía la curva de aprendizaje y permitía enfocarse más en los aspectos técnicos del aseguramiento de calidad que en aprender la metodología misma.

También se consideró que Scrum era compatible con la incorporación gradual de prácticas de XP. No había contradicción en usar Scrum como marco organizacional mientras se adoptaban prácticas técnicas específicas de XP como TDD o integración continua. Esta combinación permitiría obtener lo mejor de ambos mundos: la estructura organizacional de Scrum y las prácticas técnicas rigurosas de XP. De hecho, muchos equipos exitosos operan precisamente de esta manera, usando Scrum como contenedor y enriqueciéndolo con prácticas técnicas específicas según las necesidades del proyecto.

3.3. Datos e instrumentos

La recopilación de información combinó el registro cuantitativo proveniente del repositorio de desarrollo con la observación cualitativa del proceso de ingeniería. Esta combinación de fuentes fue deliberada y respondía a la necesidad de capturar tanto los aspectos medibles y objetivos del proceso como las percepciones, desafíos y aprendizajes del equipo que no siempre se reflejan en las métricas numéricas. La complementariedad entre datos cuantitativos y cualitativos proporcionó una visión más completa y matizada del impacto de las intervenciones en el proceso de desarrollo.

Se hizo seguimiento exhaustivo a *issues*, peticiones de cambio (*pull requests*), resultados de las ejecuciones en la *pipeline* de entrega (CI/CD), cobertura de pruebas (unitarias y de integración), el registro de defectos (antes y después de la automatización) y las métricas de rendimiento bajo cargas simuladas. Cada una de estas fuentes de datos proporcionaba una ventana diferente hacia aspectos específicos de la calidad del sistema. Los *issues* revelaban qué problemas encontraban los usuarios y desarrolladores. Las *pull requests* mostraban cómo evolucionaba el código y qué tan efectivo era el proceso de revisión. Los resultados del pipeline indicaban la frecuencia y naturaleza de los fallos detectados automáticamente.

La cobertura de pruebas medía qué tan exhaustivamente se validaba el código. El registro de defectos documentaba problemas que escapaban a las pruebas automatizadas. Y las métricas de rendimiento evaluaban cómo respondía el sistema bajo estrés.

Se instrumentaron *scripts* de extracción y limpieza de datos para análisis comparativo. Estos scripts automatizaron la tarea de recopilar información de diversas fuentes —el sistema de control de versiones, la herramienta de gestión de proyectos, los logs del pipeline, las bases de datos de pruebas— y consolidarla en un formato unificado apto para análisis. La automatización de este proceso no solo ahorró tiempo, sino que también aseguró consistencia en cómo se recopilaban y procesaban los datos a lo largo del tiempo, eliminando variaciones que podrían haber surgido si este trabajo se hiciera manualmente.

La limpieza de datos fue un paso crítico pero a menudo subestimado. Los datos crudos provenientes de sistemas diversos contenían inevitablemente inconsistencias, valores faltantes, duplicados y formatos heterogéneos. Los scripts de limpieza normalizaban fechas, estandarizaban categorías, identificaban y manejaban valores atípicos, y validaban la integridad de los datos. Sin este paso de preparación, cualquier análisis subsecuente habría estado construido sobre cimientos inestables, potencialmente llevando a conclusiones erróneas.

La recolección se extendió durante un periodo suficiente para observar patrones de regresión y la mejora progresiva asociada a la intervención en el aseguramiento de calidad. La duración de la recolección de datos fue una decisión estratégica importante. Un periodo demasiado corto podría haber capturado solo efectos transitorios o no haber permitido que las nuevas prácticas se asentaran lo suficiente como para mostrar su impacto real. Un periodo excesivamente largo, por otro lado, podría haber introducido factores confusores adicionales como cambios en el equipo, evolución significativa de los requisitos o modificaciones en la infraestructura técnica. El periodo elegido buscó el equilibrio entre dar suficiente tiempo para observar efectos genuinos y mantener otras variables relativamente constantes.

Además de los datos objetivos provenientes de sistemas, se recopilaban observaciones cualitativas a través de notas de las retrospectivas de sprint, conversaciones con miembros del equipo y documentación de decisiones técnicas importantes. Estas fuentes cualitativas capturaron el contexto, las motivaciones y los desafíos que no eran evidentes solo mirando métricas. Por ejemplo, una disminución en la cobertura de pruebas podría parecer negativa en los números, pero las notas cualitativas podrían revelar que fue una decisión consciente de eliminar pruebas redundantes o de baja calidad, lo que en realidad representaba una mejora en la estrategia de pruebas.

3.4. Procedimiento y validez

El procedimiento se articuló mediante fases claras y *checkpoints* de control, con roles específicos para planificación, ejecución de validaciones y análisis métrico. Esta estructura no fue accidental sino diseñada deliberadamente para mantener el rigor y la reproducibilidad del proceso investigativo. Cada fase tenía objetivos definidos, entregables esperados y criterios para determinar cuándo se había

completado satisfactoriamente. Los checkpoints funcionaban como puntos de decisión donde se evaluaba el progreso, se revisaban los hallazgos preliminares y se decidía si continuar según lo planeado o hacer ajustes basados en lo aprendido hasta ese momento.

La asignación de roles específicos aseguró que hubiera claridad sobre quién era responsable de qué actividades. Un rol se enfocaba en la planificación estratégica, decidiendo qué intervenciones implementar y en qué secuencia. Otro rol se concentraba en la ejecución técnica de las validaciones, asegurando que las pruebas se ejecutaran correctamente y que los resultados fueran confiables. Un tercer rol se dedicaba al análisis métrico, interpretando los datos recopilados y identificando patrones y tendencias. Esta especialización permitió que cada aspecto del proceso recibiera la atención y experiencia adecuadas, aunque también requirió coordinación cuidadosa para asegurar que todos trabajaran hacia los mismos objetivos.

La validez interna se blindó con la definición precisa de las métricas y la ejecución rigurosa y repetible de los *pipelines*. La validez interna se refiere a qué tan confiadamente se pueden atribuir los efectos observados a las intervenciones realizadas, en lugar de a otros factores. Definir las métricas con precisión eliminó ambigüedad sobre qué estaba siendo medido exactamente. Por ejemplo, en lugar de hablar vagamente de “calidad del código”, se definieron métricas específicas como densidad de defectos por mil líneas de código, cobertura de pruebas como porcentaje de líneas ejecutadas, y complejidad ciclomática promedio. Esta especificidad hizo que las mediciones fueran reproducibles y comparables a lo largo del tiempo.

La ejecución rigurosa de los pipelines aseguró que las validaciones se realizaran de manera consistente, sin variaciones introducidas por diferencias en el entorno de ejecución o en el procedimiento seguido. Cada ejecución del pipeline usaba los mismos contenedores Docker, las mismas versiones de dependencias y los mismos scripts de prueba, eliminando fuentes potenciales de variabilidad que podrían confundir los resultados. Si las métricas mejoraban, se podía tener confianza de que era debido a cambios reales en el código o las prácticas, no a diferencias en cómo se estaban midiendo las cosas.

Para la validez externa, los hallazgos se pusieron en contraste con los patrones de efectividad documentados en la literatura técnica sobre automatización en entornos críticos. La validez externa concierne la generalización de los hallazgos más allá del caso específico estudiado. ¿Los resultados observados en este proyecto serían aplicables a otros proyectos similares? Comparar los hallazgos con lo reportado en la literatura proporcionó evidencia sobre esta cuestión. Si los patrones observados eran consistentes con lo reportado en otros contextos, eso aumentaba la confianza en que los hallazgos no eran idiosincrásicos del proyecto específico sino reflejaban principios más generales.

Esta comparación con la literatura también sirvió para contextualizar los hallazgos. Si ciertas mejoras eran mayores o menores que las reportadas en estudios similares, eso invitaba a la reflexión sobre qué factores específicos del contexto podrían explicar las diferencias. Tal vez el equipo tenía más o menos experiencia previa con automatización, o el código base tenía características particulares

que facilitaban o dificultaban ciertas intervenciones. Estas reflexiones enriquecían la comprensión y proporcionaban matices importantes que no serían evidentes solo mirando los datos del proyecto aisladamente.

La validez de constructo se abordó utilizando métricas estándar de la industria (cobertura, densidad de errores, tiempo medio de fallo), mientras que la validez de conclusión se robusteció con análisis estadístico de la varianza en las métricas clave antes y después de la intervención técnica. La validez de constructo se refiere a si las mediciones realmente capturan los conceptos teóricos que se pretende medir. Usar métricas estándar y ampliamente aceptadas en la industria proporcionó confianza de que se estaban midiendo aspectos genuinos de la calidad del software, no proxies arbitrarios o irrelevantes.

La validez de conclusión, por su parte, concierne la solidez del razonamiento estadístico usado para inferir relaciones entre variables. Simplemente observar que una métrica mejoró después de una intervención no es suficiente para concluir que la intervención causó la mejora; podría haber sido coincidencia o el resultado de otros factores. El análisis estadístico de la varianza permitió determinar si las diferencias observadas eran estadísticamente significativas o podrían explicarse por variación aleatoria normal. Este rigor cuantitativo fortaleció las conclusiones y las hizo más defendibles frente al escrutinio crítico.

La metodología Scrum resultó ser la más efectiva para integrar de forma continua las herramientas de verificación. Esta conclusión no era obvia al inicio y solo se confirmó a través de la experiencia acumulada durante el proyecto. La estructura de sprints proporcionó ritmo y puntos naturales de reflexión. Las ceremonias de Scrum crearon espacios para discusiones sobre calidad y mejoras técnicas. Y la flexibilidad dentro del marco permitió adaptar las prácticas específicas a las necesidades cambiantes del proyecto sin perder la coherencia general del proceso.

Además, Scrum facilitó la comunicación y transparencia dentro del equipo y con los stakeholders. Los incrementos de producto al final de cada sprint proporcionaban evidencia tangible de progreso y puntos concretos para obtener retroalimentación. La visibilidad del trabajo pendiente en el backlog y el progreso durante el sprint ayudaba a todos a entender el estado del proyecto y las prioridades actuales. Esta transparencia también se extendió a los aspectos de calidad: cuando las pruebas fallaban o las métricas se deterioraban, era visible para todo el equipo, lo que motivaba acción correctiva oportuna en lugar de dejar que los problemas se acumularan silenciosamente.

4. Implementación técnica y prácticas de calidad

Cuando comenzó este proyecto, lo primero fue decidir cómo dividir el sistema para poder avanzar sin tropezar con nuestro propio código. No era un proyecto pequeño, así que separar el backend, el frontend y la aplicación móvil terminó siendo la única forma de mantener las cosas manejables. Al inicio, esta decisión parecía simplemente práctica: permitía que diferentes personas trabajaran en diferentes áreas sin generar conflictos constantes en el código. Sin embargo, con el tiempo quedó claro que esta separación trajo beneficios mucho más profundos de lo que se había anticipado. Cada

parte se trabajó desde un repositorio independiente y, aunque esto implica más control y coordinación entre equipos, también nos permitió detectar fallos sin afectar al resto del sistema. Si había un problema en el frontend, por ejemplo, el backend seguía funcionando normalmente y podía ser probado de forma aislada. Esta independencia se convirtió en una ventaja estratégica que facilitó tanto el desarrollo como el mantenimiento posterior.

La decisión de trabajar con repositorios separados también tuvo implicaciones en la forma de organizar el trabajo diario. Cada equipo podía avanzar a su propio ritmo, implementar sus propias mejoras y experimentar con nuevas funcionalidades sin esperar a que otras partes del sistema estuvieran listas. Esta autonomía fue especialmente valiosa durante las etapas de mayor presión, cuando varios componentes necesitaban actualizaciones simultáneas. Además, al momento de rastrear problemas o revisar el historial de cambios, resultaba mucho más sencillo identificar qué modificación había introducido un error, ya que cada repositorio contenía solo los cambios relevantes a su dominio específico.

4.1. Arquitectura del backend

El backend se armó con varias capas. No ocurrió por seguir una metodología estricta, sino por costumbre y porque ha funcionado en otros sistemas. Al principio, la estructura surgió de manera orgánica, basándose en experiencias previas y en lo que parecía más intuitivo para organizar el código. Con el paso del tiempo, esta organización demostró ser más acertada de lo que se pensaba inicialmente, especialmente cuando el proyecto comenzó a crecer y nuevas funcionalidades se fueron sumando. Las capas que usamos fueron:

- **Entity:** los modelos base, sin reglas especiales. Aquí se definen las estructuras de datos fundamentales del sistema, representando directamente las tablas de la base de datos y las relaciones entre ellas. Mantener esta capa limpia y sin lógica adicional facilitó entender qué datos manejaba el sistema.
- **Data:** todo lo relacionado con Entity Framework Core y la comunicación con SQL Server. Esta capa se encarga de las operaciones de lectura y escritura en la base de datos, implementa los repositorios y gestiona las transacciones. Al centralizar aquí toda la interacción con la base de datos, cualquier cambio en la forma de almacenar información quedaba contenido en un solo lugar, sin afectar el resto del sistema.
- **Business:** la lógica principal del sistema. Aquí se valida casi todo antes de llegar a la base de datos. Esta capa contiene las reglas de negocio, las validaciones complejas y los flujos que determinan cómo debe comportarse el sistema en diferentes situaciones. Es el corazón del backend, donde se toman las decisiones importantes sobre qué operaciones son válidas y cómo deben ejecutarse.
- **Utilities:** pequeños módulos que se repiten en varios puntos del sistema. Incluye funciones auxiliares para formatear fechas, generar códigos únicos, enviar notificaciones y otras tareas transversales que no pertenecen a una capa específica pero que son necesarias en múltiples lugares. Centralizar estas utilidades evitó duplicar código y facilitó mantener un comportamiento consistente en todo el sistema.

- **Web:** la API, escrita en ASP.NET Core. Esta capa expone los endpoints HTTP que el frontend y la aplicación móvil consumen. Se encarga de recibir las peticiones, validar los datos de entrada básicos, invocar la lógica de negocio correspondiente y devolver las respuestas en el formato adecuado. Mantener esta capa delgada, sin lógica compleja, permitió que los controladores fueran fáciles de leer y modificar.

Esta división ayudó a tener un lugar concreto para cada cosa. La capa Business terminó siendo la más revisada porque allí se escribieron las pruebas unitarias y porque concentra las reglas que se usan a diario. Cuando surgía un problema relacionado con validaciones o flujos de trabajo, se sabía exactamente dónde buscar. No era necesario revisar múltiples archivos dispersos; toda la lógica relevante estaba en un lugar conocido. No es una arquitectura sofisticada, pero sí lo bastante ordenada como para mantenerla a largo plazo.

Además, esta organización en capas facilitó la incorporación de nuevos desarrolladores al proyecto. Al tener una estructura clara y predecible, las personas nuevas podían entender rápidamente dónde debían hacer cambios y qué impacto tendrían esos cambios. La curva de aprendizaje se redujo considerablemente, y los errores típicos de no saber dónde colocar cierta funcionalidad prácticamente desaparecieron. Cada capa tenía un propósito bien definido, y eso hacía que las decisiones sobre dónde agregar código nuevo fueran más evidentes.

4.2. Frontend y aplicación móvil

El frontend se desarrolló en Angular y la aplicación móvil en React Native. Se trabajaron de manera independiente, y solo se comunican con el backend a través de la API. Esta separación no solo fue técnica, sino que también influyó en la dinámica del equipo. Los desarrolladores del frontend podían concentrarse en mejorar la experiencia de usuario sin preocuparse por los detalles de cómo se procesaban los datos en el servidor. De forma similar, los desarrolladores del backend podían optimizar la lógica y el rendimiento sin tener que coordinarse constantemente con los cambios visuales de las interfaces.

Esta forma de trabajar facilitó mucho las pruebas locales, porque permitía simular llamadas a la API sin necesidad de levantar todo el sistema. Durante el desarrollo, era común usar herramientas para interceptar las peticiones HTTP y devolver respuestas predefinidas, lo que aceleraba el ciclo de desarrollo y permitía probar diferentes escenarios sin depender de que el backend estuviera disponible. Esta capacidad de trabajar de forma desacoplada resultó invaluable cuando había que solucionar problemas urgentes o implementar cambios rápidos en la interfaz.

En estas dos capas se hicieron algunas pruebas unitarias, enfocadas solamente en los flujos que el usuario realiza con más frecuencia. El objetivo no era una cobertura amplia, sino tener un filtro mínimo antes de generar un build. Se priorizaron las pruebas de componentes críticos como los formularios de autenticación, la búsqueda de disponibilidad y el proceso de confirmación de reservas. Estos flujos eran los que más interacción tenían con el usuario final y, por lo tanto, los que más riesgo representaban si algo fallaba. Aunque las pruebas no cubrían todos los casos posibles, sí

proporcionaban una red de seguridad básica que alertaba cuando un cambio rompía alguna funcionalidad esencial.

La elección de Angular para el frontend y React Native para la aplicación móvil también respondió a consideraciones prácticas. Angular ofrecía una estructura robusta y opinada que facilitaba mantener el código organizado a medida que la aplicación crecía. React Native, por su parte, permitía compartir código entre las versiones de iOS y Android, reduciendo significativamente el esfuerzo de desarrollo y mantenimiento. Ambas tecnologías tenían comunidades activas y abundante documentación, lo que facilitaba resolver problemas y encontrar soluciones a desafíos comunes.

4.3. Integración de DevOps

El proyecto utilizó Jenkins desde etapas tempranas. Fue una decisión práctica porque evitó que cada despliegue dependiera de comandos manuales. Al principio, configurar Jenkins requirió cierto esfuerzo y familiarizarse con su funcionamiento, pero la inversión valió la pena rápidamente. Una vez que los pipelines estaban funcionando, el proceso de construcción y despliegue se volvió predecible y confiable. Todos los entornos terminaban pasando por un pipeline, aunque el destino de cada uno era distinto. Esta consistencia en el proceso eliminó una fuente importante de errores y permitió que el equipo se enfocara en desarrollar funcionalidades en lugar de lidiar con problemas de despliegue.

La automatización también tuvo un efecto positivo en la moral del equipo. Antes de tener Jenkins, cada despliegue era un evento estresante que requería atención cuidadosa y coordinación entre varias personas. Con los pipelines automatizados, desplegar se convirtió en una tarea rutinaria que cualquiera podía ejecutar con confianza. Esta reducción en el estrés y la carga cognitiva liberó energía que el equipo pudo dedicar a tareas más creativas y valiosas.

4.3.1. Entornos locales. Para *dev*, *qa* y *staging*, los pipelines realizaban:

1. Compilación de backend y frontend. Este paso aseguraba que el código se pudiera construir correctamente y que no hubiera errores de sintaxis o dependencias faltantes. Si la compilación fallaba, el pipeline se detenía inmediatamente, evitando que código defectuoso avanzara a etapas posteriores.
2. Ejecución de las pruebas unitarias. Aunque la cobertura era limitada, las pruebas existentes se ejecutaban automáticamente en cada construcción. Esto proporcionaba una validación básica de que los cambios recientes no habían roto funcionalidades conocidas. Si alguna prueba fallaba, el equipo era notificado de inmediato y podía corregir el problema antes de que se propagara.
3. Construcción de las imágenes Docker. Empaquetar la aplicación en contenedores Docker garantizaba que el entorno de ejecución fuera consistente en todos los ambientes. Las imágenes incluían todas las dependencias necesarias, eliminando el clásico problema de "funciona en mi máquina pero no en el servidor". Esta consistencia facilitaba identificar y resolver problemas, ya que se eliminaban las variables relacionadas con diferencias en el entorno.

4. Ejecución de los contenedores en local. Una vez construidas las imágenes, los contenedores se levantaban automáticamente en el entorno correspondiente. Esto hacía que el sistema estuviera disponible para pruebas manuales en cuestión de minutos después de hacer un cambio. La rapidez del proceso permitía ciclos de retroalimentación cortos, lo que aceleraba el desarrollo y facilitaba la detección temprana de problemas.

Esto hacía que las pruebas manuales fueran más rápidas y que todos los miembros del equipo trabajaran sobre entornos similares. La consistencia entre equipos apareció casi de forma natural al automatizar esas tareas. Ya no había discusiones sobre qué versión de una dependencia estaba instalada en cada máquina o sobre diferencias sutiles en la configuración. Todos trabajaban con los mismos contenedores, construidos de la misma manera, lo que eliminaba una categoría completa de problemas relacionados con inconsistencias ambientales.

Además, tener entornos separados para desarrollo, pruebas y staging permitió un flujo de trabajo más ordenado. Los desarrolladores podían experimentar libremente en el entorno de desarrollo sin preocuparse por afectar a otros. El equipo de QA tenía un ambiente dedicado donde realizar sus verificaciones sin interferir con el trabajo de desarrollo. Y staging servía como una réplica de producción donde se podían realizar pruebas finales antes del despliegue real. Esta separación de responsabilidades y etapas contribuyó significativamente a la estabilidad del sistema.

4.3.2. Producción en AWS. Producción funcionaba distinto. Allí Jenkins tenía que publicar en la infraestructura de AWS y reiniciar el servicio. El pipeline de producción incluía pasos adicionales de verificación y requería aprobación manual antes de ejecutarse, añadiendo una capa extra de seguridad para evitar despliegues accidentales. No hubo grandes complicaciones, siempre y cuando la imagen estuviera bien construida. Este proceso redujo fallos al momento del despliegue y, sobre todo, evitó depender de accesos directos al servidor.

El uso de AWS proporcionó beneficios adicionales en términos de escalabilidad y confiabilidad. Los servicios gestionados de AWS simplificaron tareas como el balanceo de carga, el monitoreo y las copias de seguridad. Aunque el proyecto no llegó a explotar todas las capacidades de la plataforma, tener esa infraestructura disponible dejó el camino abierto para mejoras futuras. La posibilidad de escalar horizontalmente agregando más instancias o de implementar esquemas de alta disponibilidad estaba al alcance sin necesidad de cambios arquitectónicos fundamentales.

Además, el hecho de que Jenkins manejara los despliegues a producción significaba que había un registro completo de cuándo se había desplegado qué versión, quién había aprobado el despliegue y qué cambios incluía. Esta trazabilidad resultó invaluable cuando era necesario investigar problemas o entender la evolución del sistema a lo largo del tiempo. Cada despliegue quedaba documentado automáticamente, sin depender de que alguien recordara actualizar manualmente un registro.

4.4. Pruebas unitarias

Las pruebas que se hicieron en el backend se centraron en la capa Business. No se trató de una cobertura grande, sino de validar algunas reglas que suelen ser las que primero fallan cuando algo cambia. Por ejemplo, revisar que los horarios estuvieran bien calculados o que el proceso básico para crear una cita no se rompiera con modificaciones recientes. Estas pruebas actuaban como una red de seguridad básica que alertaba cuando cambios aparentemente inocuos tenían consecuencias inesperadas en la lógica de negocio.

La decisión de enfocarse en la capa Business tuvo sentido porque era allí donde residía la mayor complejidad del sistema. Las reglas para validar disponibilidad, gestionar conflictos de horarios y aplicar políticas de cancelación eran intrincadas y propensas a errores sutiles. Tener pruebas que verificaran estos comportamientos críticos proporcionaba confianza al hacer cambios. Aunque las pruebas no cubrían todos los casos extremos, sí cubrían los escenarios más comunes y los que tenían mayor probabilidad de causar problemas visibles para los usuarios.

En el frontend y en la aplicación móvil no se avanzó mucho con las pruebas. Solo se cubrieron algunas acciones que los usuarios repetían con frecuencia. Cosas como iniciar sesión, buscar horarios y reservar. Era lo que más riesgo tenía de romperse con un cambio pequeño, así que se priorizó eso. La filosofía fue pragmática: en lugar de intentar una cobertura exhaustiva que consumiría mucho tiempo, se optó por proteger los flujos críticos que, si fallaban, impedirían a los usuarios completar sus tareas principales.

Esta estrategia de probar primero lo esencial también reflejaba las limitaciones de recursos y tiempo del proyecto. No había capacidad para escribir y mantener miles de pruebas, así que había que ser selectivos. La pregunta que guiaba las decisiones era: si esta funcionalidad falla, ¿qué tan grave es el impacto para el usuario? Las funcionalidades con alto impacto recibían pruebas; las de bajo impacto se dejaban para más adelante. Este enfoque no era ideal desde una perspectiva purista, pero era realista y práctico dadas las circunstancias del proyecto.

4.5. Aspectos pendientes

Mientras se avanzaba con el proyecto fueron apareciendo varios puntos que valía la pena revisar con más calma. Uno de ellos era probar cómo respondía la API cuando la aplicación completa hacía las solicitudes reales, no solo funciones sueltas. Las pruebas unitarias validaban componentes aislados, pero no garantizaban que esos componentes funcionaran bien juntos. Un error típico era que el backend devolviera datos en un formato ligeramente diferente al que el frontend esperaba, causando fallos que solo aparecían cuando ambas partes interactuaban. Pruebas de integración habrían detectado estos problemas tempranamente, antes de que llegaran a los usuarios.

También quedó pendiente ver el comportamiento de la base de datos bajo situaciones más parecidas a lo que pasa en producción, porque algunas fallas solo aparecen cuando se juntan todos los componentes. Por ejemplo, problemas de concurrencia que surgen cuando múltiples transacciones intentan modificar los mismos registros simultáneamente, o consultas que se vuelven lentas cuando la cantidad de datos crece. Estos escenarios son difíciles de simular

con pruebas unitarias, pero son realidades que el sistema enfrentaría inevitablemente en uso real.

También está pendiente validar un flujo completo desde la interfaz hasta el backend. En la práctica, muchos errores aparecen cuando todo el recorrido se une, no cuando se prueba cada paso por separado. Un flujo completo involucra navegación en la interfaz, validaciones del lado del cliente, llamadas HTTP, procesamiento en el backend, acceso a la base de datos y respuestas que vuelven hasta la interfaz. Cada una de estas etapas puede funcionar perfectamente en aislamiento, pero fallar cuando se conectan debido a suposiciones incompatibles o malentendidos sobre el contrato entre componentes.

Y otro tema es la carga: cuando hay muchos pacientes intentando reservar citas al mismo tiempo, el sistema debería resistir bien, pero eso no se ha evaluado de forma formal. Las pruebas de carga son especialmente importantes para sistemas de reservas, donde pueden existir momentos de alta demanda predecibles, como cuando se abren nuevos horarios para un médico popular o cuando hay campañas institucionales que dirigen tráfico al sistema. Sin conocer los límites del sistema, es imposible planificar adecuadamente la infraestructura o implementar estrategias de mitigación como colas o limitación de tasa.

Integrar cualquiera de estas pruebas dentro del pipeline actual sería útil. El sistema ya usa Jenkins, así que técnicamente sería posible automatizar gran parte de ese trabajo sin cambios grandes. La infraestructura básica está en su lugar; solo faltaría configurar las pruebas adicionales y conectarlas al proceso existente. Esto no requeriría una reescritura del sistema ni inversiones masivas en nuevas herramientas. Sería más bien una extensión natural de lo que ya funciona, aprovechando la base establecida para llevar el aseguramiento de calidad al siguiente nivel.

4.6. Resumen

En general, el proyecto dejó una base estable. No es perfecta, pero sí lo suficientemente ordenada como para seguir construyendo. La arquitectura en capas proporciona estructura, la separación entre frontend, backend y aplicación móvil facilita el trabajo paralelo, y el pipeline de despliegue automatizado reduce errores y da confianza. El flujo de despliegue ya funciona y las pruebas unitarias, aunque limitadas, marcaron un primer paso hacia un proceso más seguro.

Lo que sigue es ampliar esa base, agregar más tipos de pruebas y aprovechar mejor la automatización que ya existe. Las pruebas de integración, las pruebas de extremo a extremo y las pruebas de carga son los siguientes pasos lógicos. Cada una de estas adiciones haría el sistema más robusto y daría mayor visibilidad sobre su comportamiento real. El sistema está en un punto donde avanzar hacia un aseguramiento de calidad más completo no es complicado; solo requiere tiempo y priorización.

También vale la pena reconocer que el proyecto llegó donde está gracias a decisiones pragmáticas tomadas bajo presión y con recursos limitados. No siguió un plan maestro perfecto diseñado desde el inicio, sino que evolucionó orgánicamente respondiendo a necesidades reales. Y precisamente porque se tomaron decisiones razonables en el camino, el sistema tiene ahora una base sólida sobre la cual construir. El siguiente paso es tomar esas buenas decisiones

del pasado y continuarlas hacia el futuro, invirtiendo en las áreas que más impacto tendrán en la calidad y confiabilidad del sistema.

5. Resultados del análisis

En este apartado se recogen los resultados que surgieron al comparar la experiencia de trabajo en el proyecto con lo que dicen distintos autores sobre automatización, calidad del software y manejo de entornos. No se trata de resultados numéricos ni de una medición formal; es más bien una lectura técnica de lo que ocurrió y de lo que podría haber pasado si el sistema hubiese contado con un conjunto más amplio de pruebas.

5.1. Coincidencias con lo que menciona la literatura

Al revisar los documentos y artículos relacionados con automatización, una de las cosas que más llamó la atención fue que varias decisiones que se tomaron en el proyecto —sin pensarlo demasiado— coinciden con prácticas que se consideran positivas. Por ejemplo, la separación entre backend, frontend y móvil no se adoptó siguiendo un manual, sino porque era la opción más razonable para avanzar sin estorbarse mutuamente. Curiosamente, muchos trabajos mencionan que esta separación favorece la estabilidad y hace más sencillo el proceso de probar partes del sistema sin tener que levantar todo lo demás.

Lo mismo sucede con la estructura interna del backend. Aunque no se aplicó un patrón estricto, la división terminó permitiendo que las pruebas unitarias se centraran en la capa Business sin tener que depender de la base de datos. Esa independencia, según varios estudios, es un paso importante para lograr una validación más limpia de la lógica.

5.2. Lo que la literatura sugiere y el proyecto no alcanzó a cubrir

En este punto aparecieron las diferencias más notables. Casi toda la bibliografía que se revisó plantea que un sistema robusto debe tener más de un tipo de prueba. La mayoría coinciden en lo mismo: las pruebas unitarias solo cubren una parte pequeña del comportamiento real del sistema y no garantizan que todo funcione bien cuando las piezas se conectan entre sí. En el proyecto solo se llegó a ese nivel básico.

No hubo pruebas de integración completas. Tampoco se realizaron pruebas end-to-end para asegurarse de que un flujo real —como iniciar sesión, elegir un horario y finalizar una reserva— funcionara de principio a fin sin intervención humana. Al revisar los artículos, es evidente que estas pruebas son las que más ayudan a detectar fallos en el recorrido general del usuario.

Otro punto donde el proyecto quedó corto fue en el rendimiento. Ninguna parte del desarrollo incluyó mediciones de carga y tampoco se simuló un escenario con varios usuarios intentando reservar citas en un mismo momento, lo que es bastante común en este tipo de sistemas. La literatura recalca que este tipo de pruebas no solo ayuda a medir rendimiento, sino que también permite anticipar comportamientos inesperados cuando el sistema está bajo presión.

5.3. Efecto del pipeline en los resultados reales

Una de las conclusiones más evidentes fue que el pipeline sí aportó beneficios. Aunque el sistema no tenía pruebas avanzadas, la automatización de los despliegues permitió evitar buena parte de los errores que suelen aparecer cuando cada integrante del equipo despliega manualmente. Varios artículos mencionan que este tipo de automatización, incluso si no está completa, ayuda a que el equipo tenga más control sobre las versiones que llegan a producción.

Esta coincidencia fue clara: el proyecto se benefició de la constancia del pipeline, del uso de Docker y del despliegue controlado en los entornos locales y en AWS. No fue un proceso perfecto, pero sí evitó varios problemas que podrían haber aparecido si todo se hubiera manejado sin automatización.

5.4. Lo que podría haber cambiado con más pruebas

Al imaginar cómo habría sido el proyecto con un sistema de pruebas más completo, surgen varios escenarios posibles. Por ejemplo, habría sido más fácil detectar cuando los horarios disponibles no coincidían con la lógica que usaban los médicos para asignar citas. Lo mismo con errores en la reserva: con pruebas de extremo a extremo, esos fallos habrían aparecido justo después de un cambio y no varios pasos después, cuando el sistema ya estaba más avanzado.

También se habría logrado entender mejor el comportamiento del sistema en situaciones con mucha gente conectada. Esto habría permitido ajustar consultas, optimizar endpoints o incluso modificar reglas de negocio para evitar saturar la base de datos.

5.5. Balance general

Después de analizar los puntos fuertes y los débiles, un resultado claro es que el sistema tiene una base sólida en cuanto a estructura y despliegue, pero todavía no cuenta con suficientes herramientas para detectar errores sin intervención humana. Lo positivo es que no se necesita reconstruir nada; el sistema tiene la separación y la organización necesarias para incorporar pruebas más completas.

Es decir, no se está empezando desde cero. Ya hay un camino recorrido que encaja bien con lo que recomiendan los estudios revisados. A partir de aquí, integrar pruebas de integración, end-to-end y de rendimiento permitiría que el sistema crezca sin perder control sobre la calidad.

6. Discusión

Cuando se terminó la revisión del proyecto y se comparó con lo que dicen distintos autores sobre automatización y calidad, aparecieron varios temas que no se habían visto con tanta claridad durante el desarrollo. Es distinto trabajar bajo presión, tratando de que todo funcione para el usuario, que detenerse después a revisar con calma qué decisiones fueron acertadas y qué cosas se pudieron hacer distinto. Durante el desarrollo, las prioridades estaban puestas en resolver los problemas inmediatos: que la interfaz respondiera, que los datos se guardaran correctamente, que el usuario pudiera completar su flujo sin interrupciones. Ahora, con cierta distancia, es posible ver el proyecto desde otro ángulo y reconocer tanto los aciertos como las oportunidades que quedaron sin aprovechar. En

esta sección se comentan esas observaciones, sin intentar encajarlas en un esquema rígido, porque la experiencia real del proyecto no avanzó de forma tan ordenada como lo hacen los modelos teóricos.

6.1. Decisiones arquitectónicas y su alineación con la literatura

Una de las primeras impresiones al revisar la literatura es que, sin planearlo, varias decisiones del proyecto terminaron coincidiendo con lo que se recomienda hoy en día. La separación entre backend, frontend y la aplicación móvil fue más una respuesta al día a día que un intento por seguir un estándar. En un principio, la separación surgió porque diferentes personas estaban trabajando en diferentes partes del sistema, y parecía natural que cada quien tuviera su espacio. Sin embargo, cuando se observa con un poco más de distancia, es evidente que esa separación evitó muchos problemas. Permitió trabajar sin pisarse el código y facilitó entender dónde estaba fallando algo. Si todo hubiera estado en un solo proyecto monolítico, cualquier cambio habría requerido coordinación constante y habría aumentado el riesgo de romper funcionalidades ajenas.

Los estudios revisados señalan que esta independencia entre componentes es una de las bases para construir sistemas más estables, y en ese sentido, la práctica coincidió con la teoría. La modularidad no solo ayudó durante el desarrollo, sino que también simplificó las tareas de mantenimiento. Cuando surgía un problema en la interfaz móvil, no era necesario revisar todo el backend para encontrar la causa. Del mismo modo, cuando se necesitaba ajustar la lógica de negocio, no había que preocuparse por los efectos en el diseño visual del frontend. Esta independencia técnica terminó siendo una ventaja estratégica que, en su momento, no se valoró con toda la dimensión que merecía.

6.2. Estructura interna del backend y organización en capas

También se puede decir algo parecido de la arquitectura interna del backend. No se diseñó siguiendo un patrón académico específico, pero la organización en capas terminó ayudando más de lo esperado. Cuando el proyecto avanzaba rápido, esta estructura evitó que todo terminara mezclado. Al principio, la organización en capas respondía más a una intuición de orden que a un plan detallado. Sin embargo, con el tiempo, quedó claro que tener la lógica de negocio separada de los controladores y de la capa de acceso a datos hacía todo más comprensible. Cada capa tenía un propósito claro, y eso reducía la cantidad de lugares donde buscar cuando algo fallaba.

La capa Business, por ejemplo, terminó convirtiéndose en el punto donde se podían concentrar las pruebas unitarias sin depender de la base de datos. Esto fue especialmente útil cuando se necesitaba probar validaciones complejas o reglas de negocio que involucraban múltiples condiciones. Y aunque estas pruebas fueron pocas, sí permitieron detectar errores que habrían sido más difíciles de encontrar si la lógica estuviera dispersa. Además, esta separación facilitó la lectura del código para nuevos colaboradores o para revisar funcionalidades después de semanas sin tocarlas. La

claridad estructural terminó siendo tan valiosa como la funcionalidad misma.

Otro beneficio menos evidente de esta organización fue la posibilidad de hacer cambios localizados. Cuando se decidió modificar la forma en que se almacenaban ciertos datos, solo fue necesario ajustar la capa de acceso a datos sin tocar la lógica de negocio. De igual manera, cuando hubo que cambiar la forma en que se validaban algunos campos, la modificación quedó contenida en la capa Business. Esta flexibilidad no fue diseñada desde el inicio, pero resultó ser una consecuencia natural de mantener las responsabilidades separadas.

6.3. Lo que faltó: el vacío en las pruebas automatizadas

Ahora bien, cuando se comparan estas decisiones con todo lo que plantea la literatura, también se puede ver con claridad lo que faltó. Casi todos los autores revisados insisten en que las pruebas de software deben abarcar más que funciones aisladas. Hablan de pruebas de integración, pruebas de recorrido completo, simulaciones de carga, revisiones automáticas del comportamiento del sistema bajo distintas condiciones. Mencionan la importancia de validar que los componentes no solo funcionen por separado, sino que también se comuniquen correctamente entre sí. Describen escenarios donde un cambio pequeño en una parte del sistema provoca fallos inesperados en otra, y explican cómo las pruebas automatizadas pueden detectar estos problemas antes de que lleguen al usuario final.

En el proyecto no se llegó a implementar nada de eso. Las pruebas unitarias cubrieron lo mínimo indispensable, pero nunca se evaluó cómo interactuaban todas las piezas juntas. No hubo pruebas que validaran el flujo completo desde que el usuario abría la aplicación hasta que completaba una reserva. Tampoco se probó qué pasaba cuando varios usuarios intentaban acceder al mismo recurso simultáneamente. Este vacío explica por qué algunos errores aparecían tarde y por qué ciertos fallos solo se descubrían cuando alguien recorría manualmente el sistema.

La ausencia de pruebas de integración significaba que cada componente podía estar funcionando correctamente en aislamiento, pero no había garantía de que funcionaran bien al conectarse. Por ejemplo, el backend podía estar devolviendo los datos en el formato esperado, pero si el frontend interpretaba ese formato de manera diferente, el error solo aparecía cuando se probaba todo junto. Y como estas pruebas no estaban automatizadas, dependían de que alguien dedicara tiempo a recorrer el sistema manualmente, algo que no siempre ocurría con la frecuencia necesaria.

6.4. El problema de la dependencia en pruebas manuales

Otro punto que surgió al comparar el proyecto con la literatura es el riesgo de depender demasiado de las pruebas manuales. Mientras el proyecto avanzaba, esto se veía como parte normal del trabajo: cambiar algo, construir el sistema, revisar la interfaz, probar que todo respondiera. Parecía lógico que alguien verificara visualmente que los botones funcionaran, que los formularios se enviaran

correctamente, que los mensajes de error aparecieran cuando debían. Sin embargo, al leer a los autores que analizan este problema, se entiende mejor por qué esto limita el crecimiento del sistema.

Las pruebas manuales no solo toman tiempo, sino que dependen de la atención y del criterio de la persona que las realiza. Eso hace que algunos errores pasen desapercibidos, especialmente los que no ocurren siempre. Un error intermitente, que aparece solo bajo ciertas condiciones o en momentos específicos, puede ser difícil de reproducir cuando se prueba manualmente. Además, a medida que el sistema crece y se agregan más funcionalidades, el esfuerzo necesario para probar todo manualmente aumenta de forma desproporcionada. Lo que al principio tomaba unos minutos, eventualmente puede convertirse en horas de trabajo repetitivo.

La literatura también señala que las pruebas manuales son propensas a la inconsistencia. Una persona puede revisar ciertos aspectos un día y otros aspectos otro día, dependiendo de su estado de ánimo, del tiempo disponible o de lo que considere más importante en ese momento. Esta variabilidad hace que la calidad del sistema dependa más de factores humanos que de un proceso sistemático y repetible. En contraste, las pruebas automatizadas siempre ejecutan las mismas verificaciones de la misma manera, sin importar cuántas veces se corran.

6.5. El valor de la automatización del despliegue

A pesar de esto, hay un aspecto del proyecto que la literatura valora mucho: la automatización del despliegue. Los pipelines en Jenkins hicieron que las actualizaciones fueran más consistentes. Evitaron errores simples como subir una versión incompleta, olvidar una dependencia o mover una configuración de un entorno a otro. Antes de implementar estos pipelines, cada despliegue era un proceso manual que requería seguir una lista de pasos, y era fácil omitir algo o ejecutar los pasos en el orden incorrecto. Con la automatización, el proceso se estandarizó y se volvió más confiable.

Los textos consultados mencionan que la automatización de despliegues reduce una categoría completa de errores y ayuda a que el equipo tenga más confianza en lo que publica. Esto fue exactamente lo que ocurrió. Incluso sin un sistema de pruebas amplio, la existencia del pipeline mantuvo los entornos funcionando de forma más predecible. Cada vez que se ejecutaba el pipeline, se tenía la certeza de que el código que llegaba a producción era el mismo que había pasado por los entornos de prueba. Esto eliminó una fuente importante de variabilidad y facilitó la identificación de problemas: si algo fallaba en producción pero no en desarrollo, la causa debía estar en las diferencias de configuración del entorno, no en el código mismo.

Además, la automatización del despliegue tuvo un efecto psicológico positivo en el equipo. Saber que el proceso de publicación era confiable redujo el miedo a hacer cambios. Antes de tener el pipeline, cada actualización generaba cierta ansiedad porque existía el riesgo de romper algo en producción. Con el pipeline funcionando, esa ansiedad disminuyó considerablemente, lo que a su vez permitió que el equipo fuera más ágil y estuviera más dispuesto a experimentar y mejorar el sistema.

6.6. Reflexiones sobre las oportunidades perdidas

Sin embargo, esta observación también lleva a una reflexión sobre lo que podría haberse logrado con más automatización. Si el pipeline hubiese incluido pruebas de integración o algún tipo de validación más profunda, el sistema habría detectado errores antes de llegar a producción o incluso antes de llegar al equipo de pruebas manuales. Es una sensación agri dulce: por un lado se reconoce que el pipeline funcionó bien, pero también deja claro que se quedó corto frente a lo que podría haber sido.

Imaginar cómo habría sido el proyecto con un conjunto completo de pruebas automatizadas no es difícil. Cada vez que alguien subiera un cambio al repositorio, el pipeline no solo construiría el sistema y lo desplegaría, sino que también ejecutaría docenas o cientos de pruebas que verificarían que todo seguía funcionando como se esperaba. Si alguna prueba fallara, el pipeline detendría el proceso y alertaría al equipo antes de que el código problemático llegara a un entorno compartido. Esto habría ahorrado incontables horas de depuración y habría permitido que el equipo se enfocara en desarrollar nuevas funcionalidades en lugar de perseguir errores.

La literatura está llena de ejemplos de equipos que transformaron su forma de trabajar al integrar pruebas automatizadas en sus pipelines. Estos equipos reportan mayor velocidad de desarrollo, menos errores en producción y equipos más satisfechos porque pasan menos tiempo apagando incendios. El proyecto en cuestión tenía todos los elementos necesarios para seguir ese camino: la arquitectura estaba organizada, el pipeline existía, y el equipo estaba dispuesto a mejorar. Solo faltaba dar el siguiente paso.

6.7. La cuestión del rendimiento y las pruebas de carga

Otro tema que aparece en la literatura y que no se abordó en el proyecto es el rendimiento. Los sistemas de reservas suelen tener momentos en los que el tráfico sube de golpe: cuando un médico habilita nuevos horarios, cuando hay un anuncio institucional, o incluso cuando varios usuarios intentan reservar a la misma hora. En estos momentos, el sistema puede comportarse de manera muy diferente a como lo hace bajo condiciones normales. Sin pruebas de carga, no hay forma de saber con precisión cómo reaccionaría el sistema ante uno de estos escenarios.

Esta falta de información no causó problemas inmediatos, pero sí deja una duda pendiente sobre la capacidad real del sistema para manejar momentos de alta demanda. ¿Qué pasaría si cien usuarios intentaran hacer una reserva al mismo tiempo? ¿Se ralentizaría el sistema? ¿Fallarían algunas transacciones? ¿Se bloquearía la base de datos? Estas preguntas no tienen respuesta porque nunca se simuló esos escenarios. La ausencia de pruebas de carga también significa que cualquier optimización de rendimiento se haría a ciegas, sin datos concretos que guiaran las decisiones.

Los autores consultados explican que las pruebas de carga no solo revelan los límites del sistema, sino que también ayudan a identificar cuellos de botella antes de que se conviertan en problemas reales. Saber que el sistema puede manejar mil usuarios concurrentes, o saber que empieza a degradarse después de quinientos, es información valiosa para la planificación y para justificar inversiones

en infraestructura. Sin esa información, el equipo está navegando sin mapa.

6.8. Balance entre lo logrado y lo pendiente

En conjunto, el análisis permite ver algo importante: el proyecto tenía buenas bases, incluso sin haberse planteado desde un enfoque de calidad estricta. La arquitectura, la separación entre componentes y la existencia del pipeline son activos que muchos proyectos no tienen. Eso facilita imaginar un crecimiento natural del sistema hacia un modelo de aseguramiento más completo. No sería necesario rediseñar nada; solo habría que ampliar las pruebas y conectar más validaciones al proceso automatizado que ya existe.

La ventaja de tener una buena base arquitectónica es que facilita la incorporación de nuevas prácticas sin tener que rehacer todo desde cero. Agregar pruebas de integración, por ejemplo, sería relativamente sencillo porque los componentes ya están desacoplados. Implementar pruebas de carga tampoco requeriría cambios estructurales, solo configurar herramientas y definir los escenarios a evaluar. El sistema está listo para evolucionar; solo necesita que alguien dé el impulso.

Además, el equipo ya tiene experiencia con la automatización gracias al pipeline de despliegue. Esa experiencia es transferible: si pudieron automatizar el despliegue, también pueden automatizar las pruebas. La curva de aprendizaje sería más suave porque ya entienden los conceptos básicos de cómo funciona la automatización y qué beneficios trae. Solo necesitarían aplicar esos mismos principios a una parte diferente del proceso de desarrollo.

6.9. Conclusión: un proyecto con potencial claro de mejora

En resumen, lo que muestra esta revisión es que el sistema avanzó bien en algunos aspectos y quedó corto en otros. No se trata de evaluar el proyecto como bueno o malo, sino de entender que, con lo que ya está construido, es posible llevarlo a un nivel más alto sin grandes reestructuraciones. Lo que hace falta es integrar pruebas que reflejen mejor el comportamiento real del sistema y aprovechar al máximo la automatización que ya está en marcha.

El camino hacia un sistema más robusto y confiable está claramente trazado. Las herramientas y las metodologías existen, la arquitectura del proyecto las puede soportar, y el equipo tiene la capacidad de implementarlas. Lo único que falta es tomar la decisión de invertir tiempo y esfuerzo en cerrar las brechas identificadas. Cada mejora que se haga en el área de pruebas automatizadas, cada nuevo escenario que se valide, cada validación que se agregue al pipeline, acercará al proyecto a los estándares que la literatura considera fundamentales para el desarrollo de software moderno.

Finalmente, esta reflexión también sirve como recordatorio de que el desarrollo de software es un proceso continuo. Ningún proyecto está terminado mientras siga en uso. Siempre habrá nuevas funcionalidades que agregar, nuevos desafíos que enfrentar, nuevas formas de mejorar. Lo importante es reconocer dónde está parado el proyecto hoy, aprender de lo que se hizo bien y de lo que se pudo hacer mejor, y usar ese conocimiento para guiar los siguientes pasos. En ese sentido, este análisis no es un punto final, sino un punto de partida para la siguiente etapa de evolución del sistema.

7. Conclusiones

Al finalizar esta revisión y volver a recorrer mentalmente lo que fue el desarrollo del sistema, surgen varias ideas que ayudan a entender mejor qué se logró, qué faltó y hacia dónde podría crecer este tipo de proyecto si se retoma en un futuro. No es una conclusión cerrada ni un listado exacto de logros y fallas, sino una mirada más honesta, basada en la experiencia propia y en lo que se encontró en la literatura.

Una de las primeras conclusiones es que la estructura del sistema fue un acierto, incluso sin haber sido diseñada con intención académica. Separar el backend, el frontend y la aplicación móvil permitió trabajar con más tranquilidad, y al final de todo, esa decisión coincidió con lo que recomiendan muchos autores. Esta forma de organización no solo facilitó identificar errores, sino que preparó al proyecto para crecer sin volverse inmanejable.

Otra idea que queda clara es que el proyecto avanzó bien en el uso de DevOps para los despliegues. El pipeline no resolvió todos los problemas, pero sí dio una base estable que evitó varios errores comunes. Automatizar la construcción de imágenes, los despliegues y la creación de contenedores hizo que el proceso fuera más confiable y menos dependiente de tareas manuales. En ese sentido, la experiencia práctica confirmó lo que señalan muchos estudios: que automatizar el despliegue es una de las formas más efectivas de mejorar la calidad del software, incluso cuando la parte de pruebas todavía no está madura.

Sin embargo, también queda claro que las pruebas automatizadas fueron insuficientes para un sistema que depende tanto del correcto funcionamiento de cada uno de sus componentes. Las pruebas unitarias que se hicieron sirvieron para detectar errores básicos y evitar regresiones pequeñas, pero no alcanzaron para validar comportamientos complejos. La literatura es muy insistente en este punto: sin pruebas de integración, sin evaluaciones de recorrido completo y sin simulaciones de carga, el sistema siempre tendrá puntos ciegos. Y este proyecto no fue la excepción.

Esto no invalida el trabajo realizado, pero sí muestra que la calidad del sistema dependió mucho del equipo y de las pruebas manuales. Esa es probablemente la limitación más grande. Cuando un sistema entra en escenarios reales, la validación manual no siempre es suficiente ni sostenible, especialmente cuando las funcionalidades o el número de usuarios comienza a crecer.

A pesar de todo lo anterior, el análisis deja una sensación positiva: no sería necesario reconstruir nada para llevar este sistema a un nivel superior de calidad. La arquitectura ya está ordenada, el pipeline ya existe, y la separación entre capas facilita implementar pruebas más amplias sin generar caos. Es decir, el proyecto no está en un punto crítico; está en un punto donde se puede evolucionar de manera natural.

Finalmente, la conclusión más valiosa quizá no sea técnica, sino práctica: este proyecto mostró que muchas veces las decisiones que parecen simples o inmediatas terminan teniendo un impacto importante en la estabilidad del sistema. También dejó claro que la automatización no es un lujo ni un complemento; es una herramienta real que puede transformar la forma en que se construye y se mantiene el software. Y aunque en este caso la automatización de pruebas no se aplicó del todo, el camino quedó abierto para hacerlo en futuras etapas.

En resumen, el trabajo realizado permite afirmar que el sistema tiene buena base, que se tomaron decisiones acertadas y que todavía hay espacio para mejorar. Las herramientas y conceptos revisados durante esta investigación servirían como guía para fortalecer la calidad, especialmente en áreas donde el proyecto avanzó menos. Lo importante es que, desde lo técnico y desde la experiencia directa, el sistema tiene las condiciones necesarias para seguir creciendo sin perder estabilidad.

8. Apéndices

Este apartado reúne varias anotaciones que se fueron acumulando durante el desarrollo y que, aunque no encajan muy bien en las secciones principales, ayudan a entender cómo se trabajó realmente. No es un apéndice tradicional lleno de tablas o código limpio; más bien son recordatorios, observaciones sueltas y pequeños extractos que se fueron guardando para no olvidarlos.

Apéndice A. Notas del backend durante el desarrollo

Una de las primeras cosas que se anotó mientras se armaba la lógica del sistema fue lo fácil que era perder de vista de dónde provenía cada dato. Por eso se dejó un pequeño archivo de notas internas para recordar qué parte del código dependía de qué modelo. No era documentación formal, solo algo como:

“Recordar que la fecha de la cita viene como local time. Si se cambia esto en el front, revisar también el validador de disponibilidad.”

Sorprendentemente, ese mensaje terminó evitando varios errores.

Otra anotación que apareció mucho era simplemente esta frase: “Revisar disponibilidad antes de reservar, siempre”. Aunque parece algo obvio, durante las primeras semanas era común olvidar ese paso cuando se hacían pruebas rápidas.

Apéndice B. Observaciones del pipeline en uso diario

En lugar de colocar aquí el pipeline escrito como código, que ya está guardado en el repositorio, se dejó registro de las sensaciones que surgieron al usarlo todos los días. Por ejemplo:

- Cuando el pipeline fallaba, casi siempre era por detalles pequeños: rutas mal escritas, faltaba una dependencia, o el contenedor anterior no se había apagado bien. - El tiempo de compilación se volvió un indicador intuitivo del estado del proyecto. Si de repente tardaba más, todos sabíamos que algo se había desordenado. - La parte de levantar los contenedores localmente terminó siendo la más confiable. Incluso cuando algo no funcionaba, al menos podíamos repetir el proceso sin depender de configuraciones manuales.

Hubo otra observación que se repitió varias veces: “El pipeline no impide errores, pero sí ahorra muchos que vienen de hacerlo todo a mano”. Esa frase resume bastante bien la utilidad que tuvo.

Apéndice C. Comentarios sobre las pruebas unitarias

Las pruebas unitarias no fueron muchas, pero dejaron algunas conclusiones que vale la pena guardar aquí.

Una de ellas fue que probar funciones pequeñas era útil, pero no suficiente. A menudo una prueba pasaba sin problemas, pero cuando el sistema completo se ponía en marcha, algo fallaba en un punto inesperado. Ese comportamiento reforzó la idea —que también aparece en la literatura— de que las pruebas aisladas sirven solo para partes muy específicas del sistema.

Otro comentario recurrente en las notas era que las pruebas ayudaban más a recordar las reglas que a detectar errores. Por ejemplo, había casos donde una prueba no fallaba, pero obligaba a revisar la regla de negocio y, al repasar esa regla, se encontraba un detalle que no estaba bien planteado.

También quedó una lista de “cosas por probar” que nunca llegó a completarse. Algunos ejemplos eran:

- “Ver qué pasa si dos usuarios reservan el mismo horario casi al mismo tiempo.” - “Probar el flujo de cambio de cita sin intervención manual.” - “Simular un día con muchos pacientes conectados.”

Esa lista sigue siendo útil hoy porque muestra las áreas que más habrían beneficiado al proyecto si se hubieran automatizado.

Apéndice D. Notas generales del equipo

Durante el desarrollo circularon varios mensajes y recordatorios que no forman parte del código, pero explican mucho sobre cómo se trabajaba. Algunos de esos apuntes quedaron guardados sin pensar que terminarían en un apéndice, pero ayudan a entender el contexto:

- “Hoy el front no responde porque el backend cambió un campo sin avisar.” - “Revisar bien las zonas horarias, en staging está una hora adelantado.” - “El contenedor funciona local pero no en QA; revisar variables de entorno.” - “No olvidar reiniciar el servicio en AWS después del despliegue.”

Estos comentarios pueden parecer simples, pero en realidad describen mejor que cualquier tabla el ambiente de trabajo y los desafíos técnicos que aparecían constantemente.

Apéndice E. Reflexiones que no cabían en otra parte

Aquí se dejó un conjunto de observaciones sueltas que aparecieron tarde en el proyecto, cuando ya se tenía más claridad sobre el sistema:

- Muchas cosas funcionaron no por diseño, sino porque se corrigieron a tiempo. - La automatización del despliegue salvó más de una entrega. - Faltaron pruebas, pero lo que se hizo permitió identificar qué partes eran más frágiles. - La separación por capas evitó que el sistema se volviera un enredo imposible de mantener.

Este tipo de anotaciones fueron útiles para escribir las conclusiones, pero también ayudan a dejar memoria técnica para futuras mejoras.

Apéndice F. Referencias complementarias

A continuación se presenta una lista de referencias externas que complementan las observaciones realizadas en las secciones anteriores, particularmente en lo referente a las **pruebas automatizadas**, **DevOps** y la **calidad del software**. Estas fuentes se incluyen únicamente como material de apoyo general y no forman parte del archivo bibliográfico procesado por *biber*.

- THE IMPACT OF AUTOMATED TESTING AND DEVOPS ON SOFTWARE QUALITY: A REVIEW
- A systematic mapping review on automated software testing: tools and challenges
- An Empirical Study of the Impact of Automated Testing on Software Quality
- A Comparative Evaluation on the Quality of Manual and Automatic Test Case Generation Techniques for Scientific Software
- A Review on the Process of Automated Software Testing
- The Next Generation of Automated Software Testing Tools
- Integrating AI in testing automation
- The Evolution of Software Testing: How AI is Redefining QA
- Software Testing from an Agile and Traditional view
- Artificial Intelligence in Software Testing: Impact, Problems, Challenges and Prospect
- Assertions in software testing: survey, landscape, and trends
- Model-based testing in practice: An experience report from the web applications domain
- The integration of machine learning into automated test generation: A systematic mapping study
- Automated Testing of Software that Uses Machine Learning APIs
- Model-Based Testing in Practice: An Industrial Case Study using GraphWalker
- A Model-Based Test Script Generation Framework and Industrial Insight
- Artificial Intelligence in Software Testing for Emerging Fields
- Expectations vs Reality — A Secondary Study on AI Adoption in Software Testing
- Contract Testing in Microservices-Based Systems: A Survey.
- An Experimental Assessment of Using Theoretical Defect Predictors to Guide Search-Based Software Testing.

Referencias